

Plural

1. What exactly is Plural PR Automation?

Plural describes PR Automation as a toolkit for automating IaC/GitOps workflows that would otherwise require developers or platform engineers to manually edit code.

It has three main patterns:

1. **Custom PR Automation CRDs**
 - Developers can generate new services/resources.
 - Developers can modify existing resources.
2. **PR Automation Pipelines**
 - PRs can be automatically generated during promotion such as:
DEV → PROD.
3. **Renovate/dependency automation**
 - Automated PRs for dependency releases such as Helm chart upgrades.

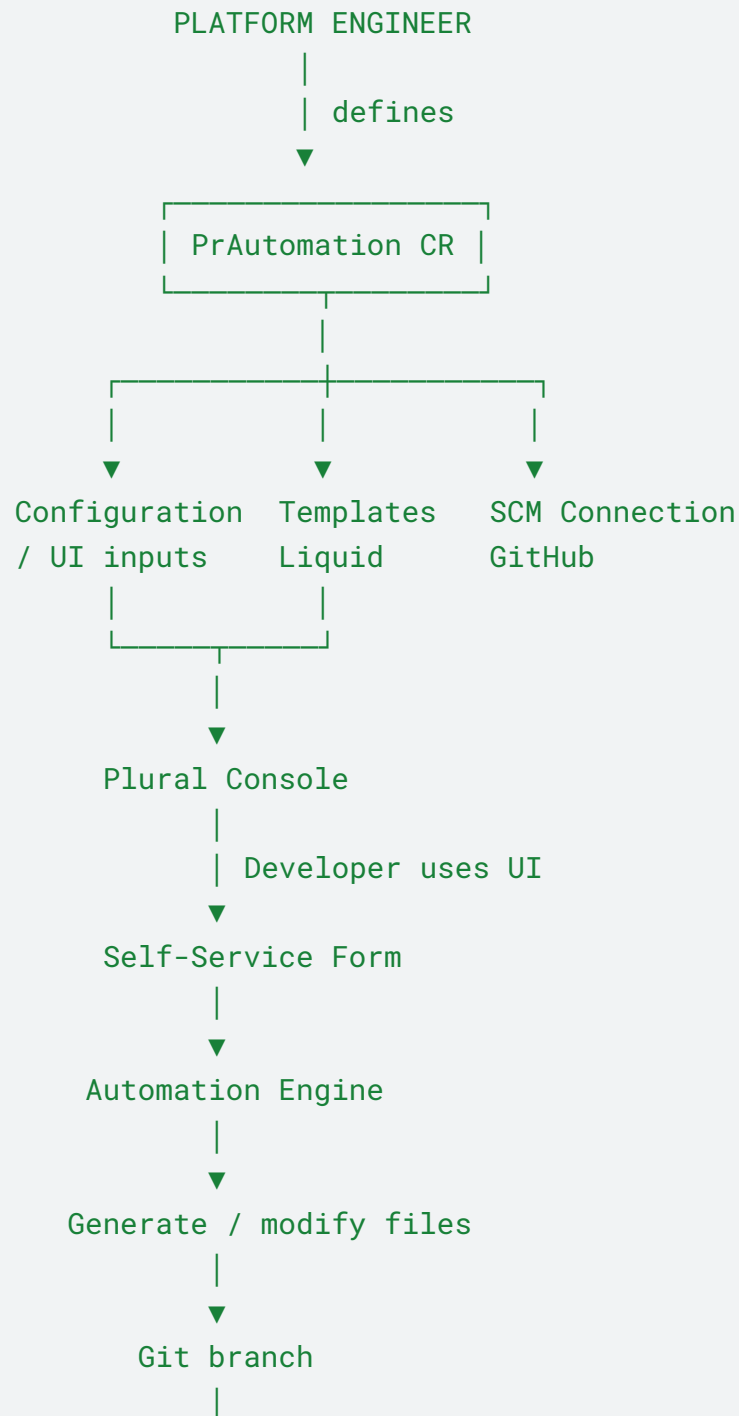
So the core idea is:

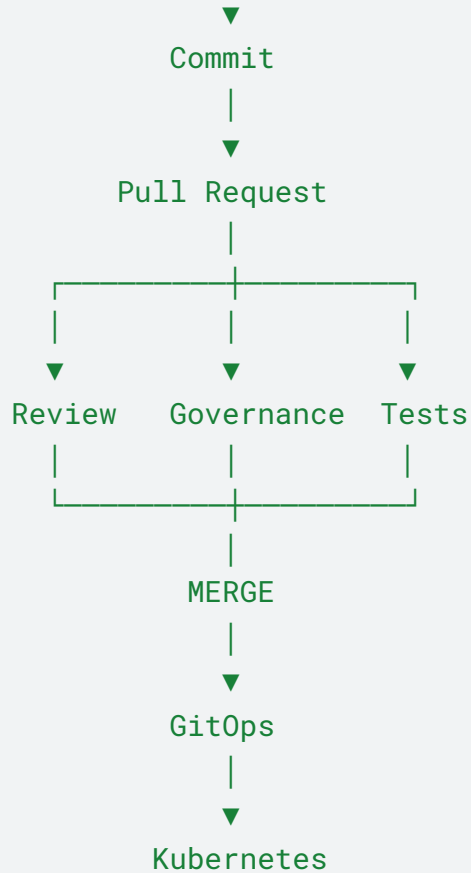


2. The most important architecture

For the normal **Self-Service PR Automation** case:

None





3. Who creates what?

This is important.

There are **two users** involved.

Platform Engineer

The platform engineer creates the:

None

PrAutomation CRD

This defines the automation.

For example:

"Create a new developer workspace."

The platform engineer decides:

- What inputs are required?
 - Which repository?
 - Which files should be generated?
 - Which templates should be used?
 - What should the PR title be?
 - What should the PR description contain?
 - Which SCM connection should be used?
-

Developer

The developer does **not** normally create the `PrAutomation`.

Instead, the developer:

```
None
opens Plural Console
  ↓
PR tab
  ↓
selects automation
  ↓
fills form
  ↓
creates PR
```

Plural explicitly says that after the automation specification is created, it becomes available in the **Plural Console under the PR tab**, where the user can create the PR using a form. crds

4. Step 1 — Configure the GitHub connection

First Plural needs to know how to communicate with GitHub.

The documentation shows:

```
None
apiVersion: deployments.plural.sh/v1alpha1
kind: ScmConnection

metadata:
  name: github

spec:
  name: github
  type: GITHUB
```

Then the PR Automation references it:

```
None
scmConnectionRef:
  name: github
```

So conceptually:

```
None
PrAutomation
  |
  | scmConnectionRef
  ▼
ScmConnection
  |
  ▼
GitHub API
```

The SCM connection is what allows Plural to perform the Git/PR operation.

5. Step 2 — Create the templates

Now the platform engineer defines **what files should be generated**.

Plural uses **Liquid templates**.

For example:

```
None
templates/
  service.yaml.liquid
  workspace.yaml.liquid
```

The first template:

```
None
apiVersion: deployments.plural.sh/v1alpha1
kind: ServiceDeployment

metadata:
  name: {{ context.name }}
  namespace: infra

spec:
  namespace: {{ context.name }}

  git:
    folder: workspaces
    ref: main

  repositoryRef:
    kind: GitRepository
    name: infra
    namespace: infra
```

```
configurationRef:
  name: plural-pull-creds
  namespace: infra

helm:
  version: "0.x.x"
  chart: workloads

valuesFiles:
  - {{ context.name }}.yaml

repository:
  namespace: infra
  name: workloads

clusterRef:
  kind: Cluster
  name: mgmt
  namespace: infra
```

Notice:

None

```
{{ context.name }}
```

That is a variable supplied by the developer through the UI.

6. Second template

The second template is:

None

```
cluster: {{ context.cluster }}

gitRepository: {{ context.repo }}

access:
  write:
    - groupName: {{ context.group }}

workloads: {{ context.workloads }}
```

This becomes the Helm values/configuration file for the workspace.

The documentation explains that the first template generates the `ServiceDeployment` CRD, while the second defines the Helm values file. The workspace can contain multiple workloads/components, effectively acting as an app-of-apps. crds

7. What is **context**?

context is basically the **developer's input data**.

For example:

None

```
{
  "name": "payments",
  "cluster": "production",
  "repo": "payments",
  "group": "backend-team",
  "workloads": "api,frontend"
}
```

Then the template can use:

None

```
{{ context.name }}
```

None

```
{{ context.cluster }}
```

None

```
{{ context.repo }}
```

etc.

So:

None

UI form

↓

context

↓

Liquid template

↓

generated YAML

This is one of the most important concepts to understand for your OpenChoreo implementation.

8. Step 3 — Define the PrAutomation

Now Plural needs to connect:

None

Templates

+

Developer inputs

```
+
GitHub
+
PR metadata
```

The CRD looks like:

```
None
apiVersion: deployments.plural.sh/v1alpha1
kind: PrAutomation

metadata:
  name: workspace-creator

spec:
  name: workspace-creator

  documentation: |
    Sets up a PR to provision a new workspace for a team.

  creates:
    templates:

      - source: templates/service.yaml.liquid
        destination: apps/workspaces/{{ context.name }}.yaml
        external: false

      - source: templates/workspace.yaml.liquid
        destination: workspaces/{{ context.name }}.yaml
        external: false

  scmConnectionRef:
    name: github

  title: 'Adding workspace {{ context.name }} for {{ context.group
  }}'
```

```
message: |
  Adding workspace {{ context.name }}
  Workloads to be provisioned:
  {{ context.workloads }}

identifier: pluralsh/pr-automation-demo

configuration:

  - name: name
    type: STRING
    documentation: The name of this workspace

  - name: cluster
    type: STRING
    documentation: The cluster this workspace deploys to

  - name: repo
    type: STRING
    documentation: The repo it is sourced from

  - name: group
    type: STRING
    documentation: A group with writer permissions

  - name: workloads
    type: STRING
    documentation: Comma separated list of workloads
```

This is the heart of the system. crds

9. What does **creates** mean?

This tells Plural:

"When this automation runs, create these files."

For example:

None

```
creates:
  templates:

    - source: templates/service.yaml.liquid
      destination: apps/workspaces/{{ context.name }}.yaml

    - source: templates/workspace.yaml.liquid
      destination: workspaces/{{ context.name }}.yaml
```

If the developer enters:

None

```
name = payments
```

Plural generates:

None

```
apps/workspaces/payments.yaml
workspaces/payments.yaml
```

So one UI action can generate multiple Git files.

10. What does **configuration** do?

This is what makes the **UI form**.

For example:

None

configuration:

- name: name
type: STRING
- name: cluster
type: STRING
- name: repo
type: STRING
- name: group
type: STRING
- name: workloads
type: STRING

Plural knows that the automation requires these values.

Therefore, when the developer selects the automation in the UI, Plural can ask for:

None

Create Workspace
Workspace Name
[payments]
Cluster
[production]
Repository
[payments-repo]
Group
[backend-team]

```
|                                     |  
| Workloads                         |  
| [ api, frontend ]               |  
|                                 |  
|           [ Create PR ]         |  
+-----+
```

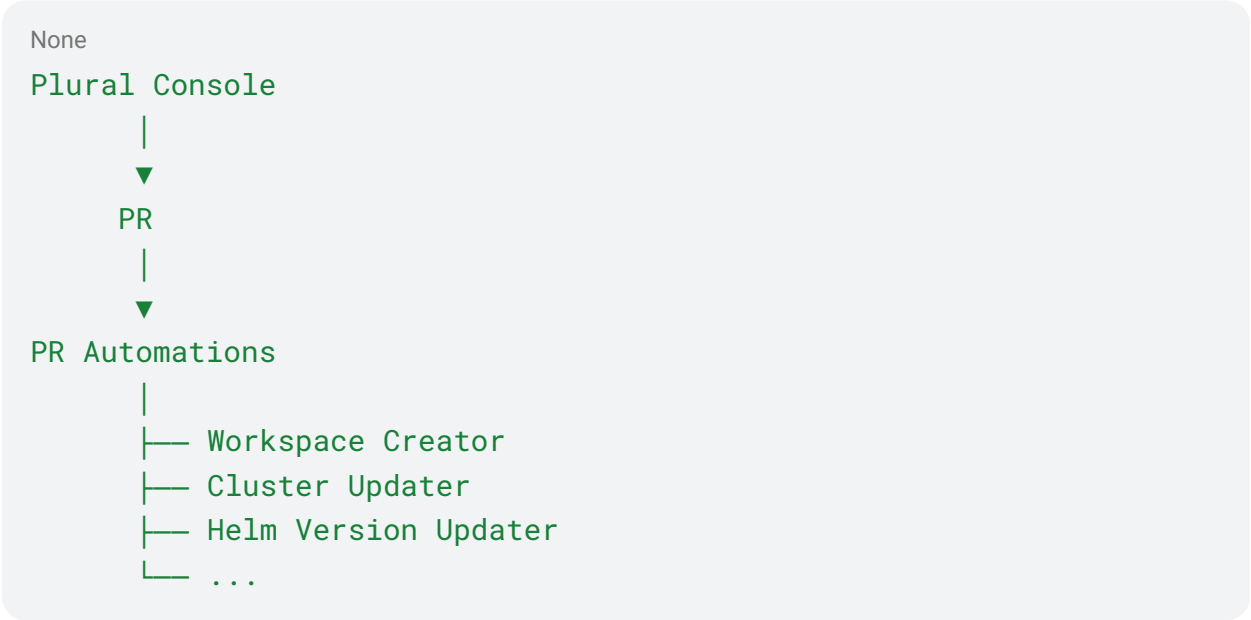
That is the key UI mechanism.

You don't need to hard-code a separate UI form for every PR workflow.

The **PrAutomation** specification describes the inputs.

11. Step 4 — Automation appears in the UI

Once the **PrAutomation** is created:



The documentation explicitly says the automation becomes available in the Plural Console under the **PR tab**. crds

12. Step 5 — Developer chooses the automation

Suppose the developer chooses:

None

Workspace Creator

Plural shows the fields defined under:

None

configuration:

The developer fills them.

For example:

None

name:

payments

cluster:

production

repo:

payments-repo

group:

backend-team

workloads:

api, frontend

13. Step 6 — Developer clicks Create PR

Now the important part starts.

Plural takes:

```
None
Developer form values
```

and converts them into:

```
None
context
```

For example:

```
None
{
  "name": "payments",
  "cluster": "production",
  "repo": "payments-repo",
  "group": "backend-team",
  "workloads": "api,frontend"
}
```

14. Step 7 — Liquid templates are rendered

Plural takes:

```
None
context
+
templates
```

and renders the final files.

For example:

```
None
metadata:
  name: {{ context.name }}
```

becomes:

```
None
metadata:
  name: payments
```

And:

```
None
cluster: {{ context.cluster }}
```

becomes:

```
None
cluster: production
```

15. Step 8 — Files are written to the configured paths

Plural has:

```
None
destination: apps/workspaces/{{ context.name }}.yaml
```

So:

None

```
context.name = payments
```

becomes:

None

```
apps/workspaces/payments.yaml
```

The second:

None

```
destination: workspaces/{{ context.name }}.yaml
```

becomes:

None

```
workspaces/payments.yaml
```

The documentation explicitly says the form results are passed to the templates and printed to the configured files. crds

16. Step 9 — Plural uses the SCM API

Now Plural has:

None

```
Repository
```

```
+
```

```
Generated files
```

```
+
```

```
PR title
```

```
+
```

```
PR message
```

```
+
```

SCM credentials

It calls the appropriate SCM API.

For the example:

None

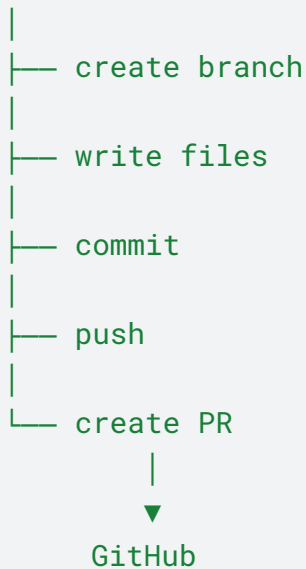
GitHub API

The documentation specifically says Plural calls the appropriate SCM API — GitHub in this example — to generate the pull request. crds

Conceptually:

None

Plural



17. What does the resulting PR look like?

Because the title is:

None

```
title: 'Adding workspace {{ context.name }} for {{ context.group }}'
```

you could get:

None

```
Adding workspace payments for backend-team
```

And the PR body might be:

None

```
Adding workspace payments
```

```
Workloads to be provisioned:  
api, frontend
```

So the PR is automatically descriptive.

18. What happens after PR creation?

This is where the architecture changes from **self-service** to **GitOps**.

None

```
Developer
```

```
↓
```

```
Plural UI
```

```
↓
```

```
PR Automation
```

```
↓
```

```
GitHub PR
```

```
↓
```

```
Code Review
```

```
↓
```

```
Approval
  ↓
Merge
  ↓
GitOps repository
  ↓
GitOps controller
  ↓
Kubernetes
```

The example assumes another service is already watching:

```
None
apps/workspaces
```

After the PR is merged, that service sees the new CRD and provisions the workspace. crds

19. Very important: Plural does NOT necessarily deploy directly from the PR

The architecture is:

```
None
PR
  ↓
Git repository
  ↓
GitOps controller
  ↓
Kubernetes
```

The PR is the **change/review mechanism**.

GitOps is the **deployment mechanism**.

That's a very important distinction.

20. PR Automation can also MODIFY existing files

The `creates` example is:

```
None
Create new workspace
```

But Plural can also modify existing configuration.

For example:

```
None
updates:
  regexReplacements:
    - regex: 'version: (.*) # VERSION'
      file: apps/pipeline/{{ context.pipeline.stage.name }}.yaml
      replacement: 'version: {{ context.version }} # VERSION'
```

Suppose:

```
None
version: 6.5.3 # VERSION
```

Developer requests:

```
None
6.5.4
```

Plural generates the Git change:

None

```
- version: 6.5.3 # VERSION
+ version: 6.5.4 # VERSION
```

and opens the PR.

This is the pattern used by the Plural PR Automation pipeline example.

21. This is why **PrAutomation** is more than a template

A **PrAutomation** can represent:

None

```
Create
Update
Delete
```

depending on the automation definition.

Conceptually:

None

```
PrAutomation
|
|— CREATE files
|
|— UPDATE files
|
|— DELETE/change resources
```

The important point is that the **developer interacts with an intent**, rather than manually editing YAML.

22. Liquid makes the automation dynamic

Plural uses Shopify Liquid.

For example:

None

```
{{ context.name }}
```

or:

None

```
{{ context.name | upper }}
```

or:

None

```
{{ context.name | lower | replace: " ", "-" }}
```

Plural provides filters for:

- strings
- mathematics
- dates
- data structures
- encoding/decoding

etc. filters

Example:

None

```
{{ context.name | lower | replace: " ", "-" }}
```

Input:

None

```
My Payment Service
```

Output:

```
None
my-payment-service
```

23. Why Liquid is useful for OpenChoreo

For your Project 702, the same concept could be:

```
None
metadata:
  name: {{ context.componentName }}

spec:
  project:
    name: {{ context.project }}

  environment:
    name: {{ context.environment }}
```

Then:

```
None
Component Name = payments
Project = ecommerce
Environment = dev
```

generates:

```
None
metadata:
  name: payments

spec:
  project:
```

```
name: ecommerce

environment:
  name: dev
```

However, **your project should continue using `occ` for OpenChoreo-aware generation rather than replacing `occ` with `Liquid`**. Your project documents specifically propose reusing `occ` to generate the correct OpenChoreo resources.

So your architecture should be:

```
None
Backstage
  ↓
Form
  ↓
context
  ↓
occ
  ↓
Workload / ComponentRelease / ReleaseBinding
  ↓
Git
  ↓
PR
```

rather than:

```
None
Backstage
  ↓
Liquid
  ↓
OpenChoreo YAML
```

24. Plural has a second PR Automation method: AI

The deterministic method is:

```
None
Input
↓
Template
↓
Known file modification
↓
PR
```

But Plural also provides AI-based PR Automation.

The documentation describes the flow as:

1. Provide a templated prompt.
2. When the automation is invoked, the sandboxed coding agent performs the required repository modification. ai-based-pr-automations

Example:

```
None
ai:
  enabled: true

  prompt: |
    Update the kubernetes version configuration
    for the {{ context.name }} cluster.

    It will be found in a file under
    clusters/{{ context.tier }}

    and should be a simple one-line YAML change.
```

The user gives:

None

```
{  
  "name": "cluster-1",  
  "tier": "prod"  
}
```

The AI performs the repository modification.

25. When does Plural use AI instead of deterministic templates?

Plural specifically positions AI PR Automation for **unstructured code changes**.

For example:

None

- Complex YAML
- Terraform
- Documentation
- Non-trivial code changes

where writing complicated regex/template logic would be inconvenient.

So:

Deterministic

None

- Known input
- +
- Known location
- +
- Known transformation

Use:

None

Liquid / regex / YAML overlays

AI

None

Less predictable repository structure

+

Complex modification

Use:

None

Prompt

+

Sandboxed coding agent

26. Plural also has Lua

There is a third mechanism.

None

Liquid

↓

simple deterministic generation

Lua

↓

complex deterministic logic

AI

↓

unstructured repository modifications

Plural's Lua integration exists for cases such as:

- generating code based on other configuration files
- content-based error handling
- automatically inferring inputs
- complex custom rules

and is sandboxed. lua

For example, Lua can:

```
None
Read images/lut.yaml
  ↓
Parse YAML
  ↓
Inspect requested images
  ↓
Find corresponding repository
  ↓
Generate YAML overlay
  ↓
Return PR Automation changes
```

So Plural has a fairly sophisticated PR generation engine.

27. Lua can even modify the automation dynamically

This is a very interesting architectural feature.

Lua can return:

```
None
prAutomation
```

or:

None

`context`

The returned `prAutomation` becomes an overlay on the original PR Automation specification.

The returned `context` can add/infer values.

That means:

None

User gives simple input



Lua derives additional information



PR Automation gets enriched



Files generated



PR

This can reduce human input and errors.

28. PR Automation Pipelines

Now we move to a different level.

Instead of:

None

Developer



PR

Plural can do:

None

Pipeline



PR Automation



PR



Next stage



PR

For example:

None

DEV



| PR Automation



DEV PR



| merge



DEV deployed



| approval gate



PROD PR



| merge



PROD deployed

The Pipeline stages point to a [PrAutomation](#).

The documentation's example has:

None

```
stages:
  - name: dev
    ...
    criteria:
      prAutomationRef:
        name: podinfo-pipeline

  - name: prod
    ...
    criteria:
      prAutomationRef:
        name: podinfo-pipeline
```

and:

None

```
edges:
  - from: dev
    to: prod
    gates:
      - name: approval-gate
        type: APPROVAL
```

So the pipeline itself decides when to invoke the PR automation.

29. How does the pipeline start?

Plural uses a:

None

PipelineContext

Example:

None

```
apiVersion: deployments.plural.sh/v1alpha1
kind: PipelineContext
```

```
metadata:
  name: podinfo-context
```

```
spec:
  pipelineRef:
    name: pr-automation
```

```
context:
  version: 6.5.4
```

Now:

None

```
version = 6.5.4
```

is passed into the PR Automation.

The pipeline then uses the stage information:

None

```
dev
```

or:

None

```
prod
```

to decide which file should be changed.

30. Pipeline flow in detail

None

PipelineContext

|
| version = 6.5.4

▼
DEV

▼
PrAutomation

▼
Modify dev.yaml

▼
Create DEV PR

▼
Review / merge

▼
DEV deployed

▼
Integration test

▼
Approval gate

▼
PrAutomation

▼
Modify prod.yaml

▼
Create PROD PR

|



This gives you **auditable automated promotion**.

31. Integration-test gates

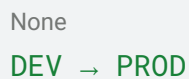
Plural can insert a Kubernetes Job as a pipeline gate.

For example:



The test can run on a specified cluster.

So the pipeline isn't simply:



It can be:



↓
test
↓
approval
↓
PROD

This is particularly relevant for enterprise deployment workflows.

32. PR Governance

Now another major part.

Plural can associate a PR with governance.

For example:

None
Plural governance: snow

means:

This PR must go through the configured ServiceNow governance process.

33. ServiceNow flow

The complete flow is:

None
Developer
↓
PR created

```
↓
Plural detects:
"Plural governance: snow"
↓
Create ServiceNow Change Request
↓
Wait for ServiceNow approval
↓
ServiceNow becomes Scheduled
↓
Plural approves PR
↓
PR merged
↓
ServiceNow → Close
```

If the PR is closed:

```
None
PR closed
↓
ServiceNow Change → Cancelled
```

The documentation specifies this lifecycle.

34. Why SCM webhook is required

Plural needs to know when something happens to the PR.

Therefore:

```
None
GitHub
|
```

```
| webhook
▼
Plural
```

The webhook provides PR/MR events.

Plural's documentation states that the SCM webhook is assumed for these PR governance/correlation mechanisms and can be configured under:

```
None
Self Service → SCM Management
```

description-patterns

35. ServiceNow governance configuration

Plural defines:

```
None
apiVersion: deployments.plural.sh/v1alpha1
kind: PrGovernance

metadata:
  name: snow

spec:
  type: SERVICE_NOW

  connectionRef:
    name: governance

  configuration:

    serviceNow:
```

```
url: https://instance.service-now.com

username: my-user

passwordSecretKeyRef:
  name: snow-creds
  key: password

secretNamespace: infra

changeModel: Standard
```

The important point is:

```
None
PrGovernance
  ↓
ServiceNow
  ↓
PR approval process
```

36. Custom enterprise governance

Plural doesn't force everyone to use ServiceNow.

You can define:

```
None
kind: PrGovernance

spec:
  type: WEBHOOK

connectionRef:
```

```
name: governance

configuration:
  webhook:
    url: https://governance-webhook.acme.com
```

Then:

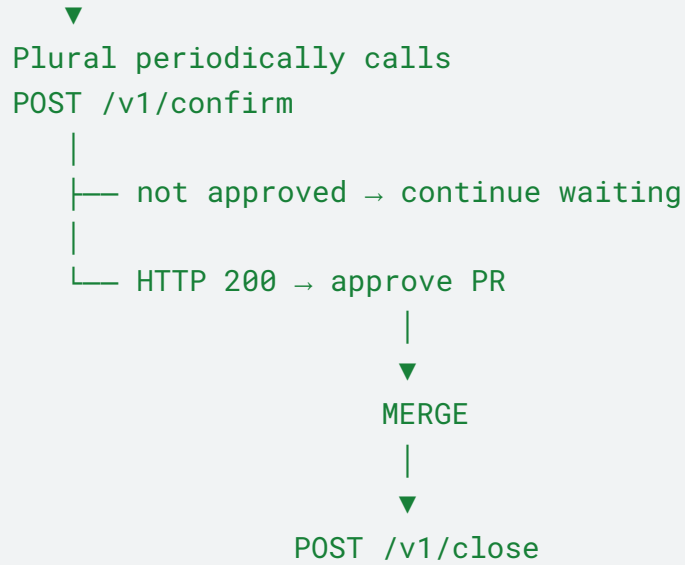
```
None
PR
↓
Plural
↓
Your governance system
```

The governance service implements:

```
None
POST /v1/open
POST /v1/confirm
POST /v1/close
```

37. Custom governance lifecycle

```
None
PR OPEN
|
▼
POST /v1/open
|
▼
Governance system creates change
|
```



The webhook receives PR information such as:

None

```
{
  "state": "...",
  "pr": {
    "id": "...",
    "title": "...",
    "body": "...",
    "ref": "...",
    "url": "...",
    "status": "open",
    "labels": []
  }
}
```

This makes the governance mechanism extensible.

38. PR correlation

Plural also puts special tags in the PR description.

For example:

Service

None

Plural Service: cluster/service

Stack

None

Plural Stack: stack-name

Governance

None

Plural governance: governance-name

Preview

None

Plural Flow: flow-name

Plural Preview: preview-template-name

These tags allow Plural to understand:

"Which Plural resource does this PR belong to?"

39. Why service correlation matters

Suppose:

None

PR #123

changes:

```
None
apps/payment/service.yaml
```

and the PR body contains:

```
None
Plural Service: prod/payment
```

Plural knows:

```
None
PR #123
  ↓
payment service
```

After merge, Plural can kick the associated service to minimize GitOps lag instead of waiting for the ordinary sync interval. description-patterns

40. Preview environments

A PR can also be linked to a preview environment:

```
None
Plural Flow: my-flow
Plural Preview: my-preview-template
```

Then:

```
None
PR opened
  ↓
Preview environment created
```

```
↓  
Developer pushes another commit  
↓  
Preview updated  
↓  
PR deleted/closed  
↓  
Preview deleted
```

So the PR becomes the lifecycle controller for the preview environment.

41. PR merge scheduling

Plural can even defer the merge.

For example:

```
None  
Plural merge cron: */5 1,2 * * *
```

means Plural can attempt to merge the PR every five minutes during the specified window, assuming the PR is already approved and the required SCM connection exists.
description-patterns

So:

```
None  
Developer creates PR  
↓  
PR approved  
↓  
wait for deployment window  
↓  
Plural auto-merges
```

This is useful for production service windows.

42. Testing PR Automations

Plural provides two important testing mechanisms.

Local testing

None

```
plural pr test --file path/to/pr-automation-cr.yaml
```

This:

None

```
reads CRD
```

↓

```
opens terminal wizard
```

↓

```
gets configuration
```

↓

```
runs same template path
```

↓

```
shows generated result
```

but:

does not create a real PR.

That lets platform engineers test the automation safely.

43. Contract testing

For CI:

None

PrAutomation

+

test context

↓

generated files

↓

compare with expected state

↓

git diff

↓

PASS / FAIL

Plural provides:

None

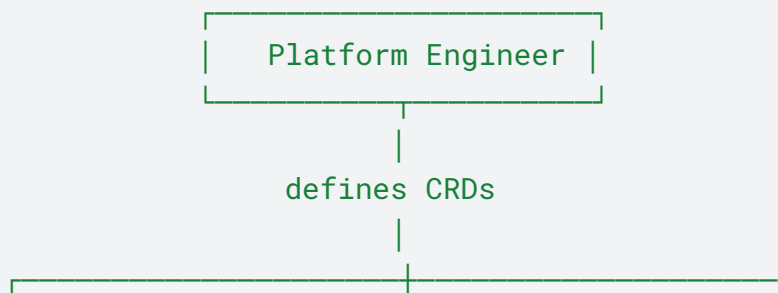
```
plural pr contracts --file contracts.yaml
```

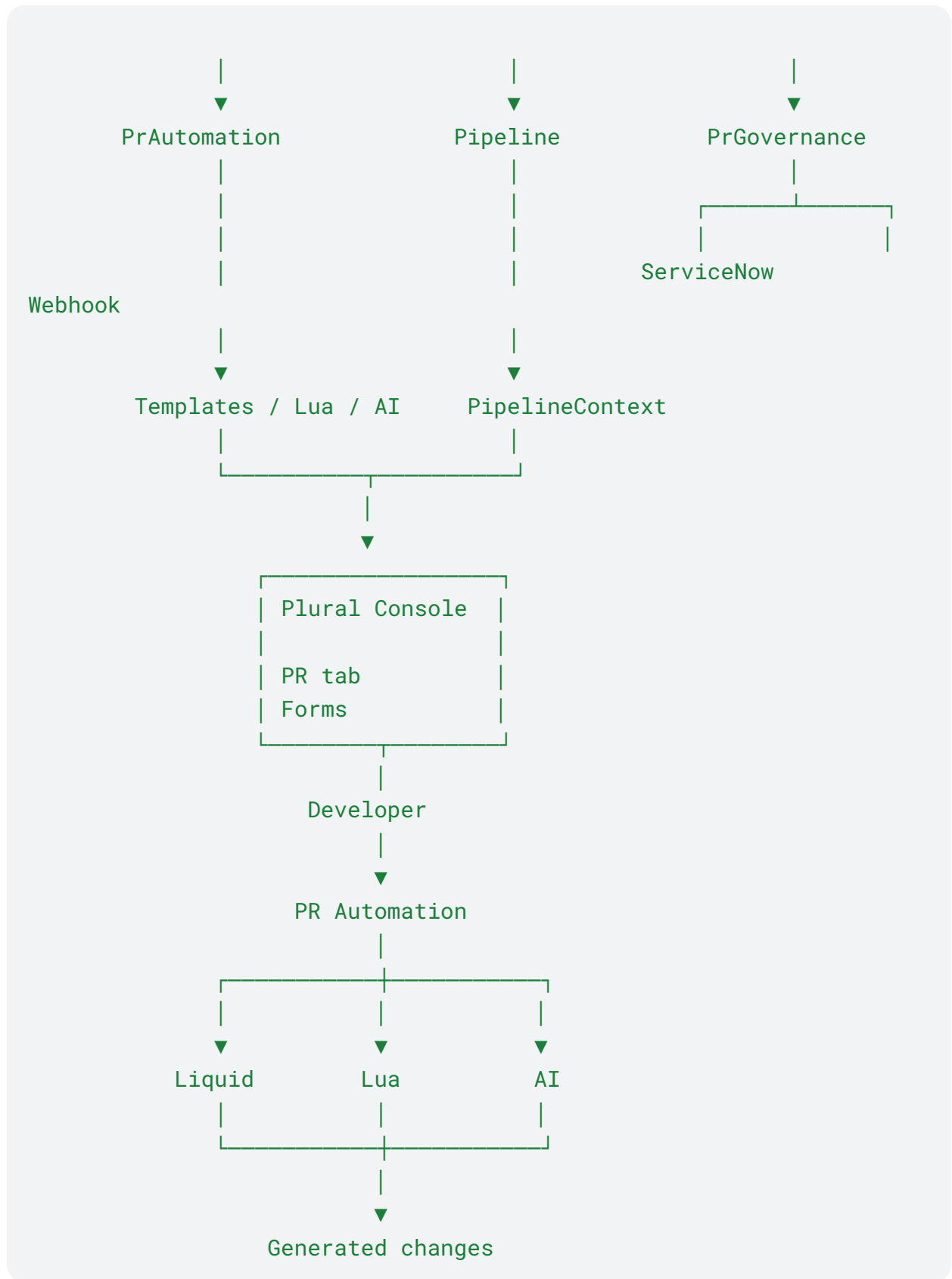
This is useful when your PR automation becomes an important platform component.

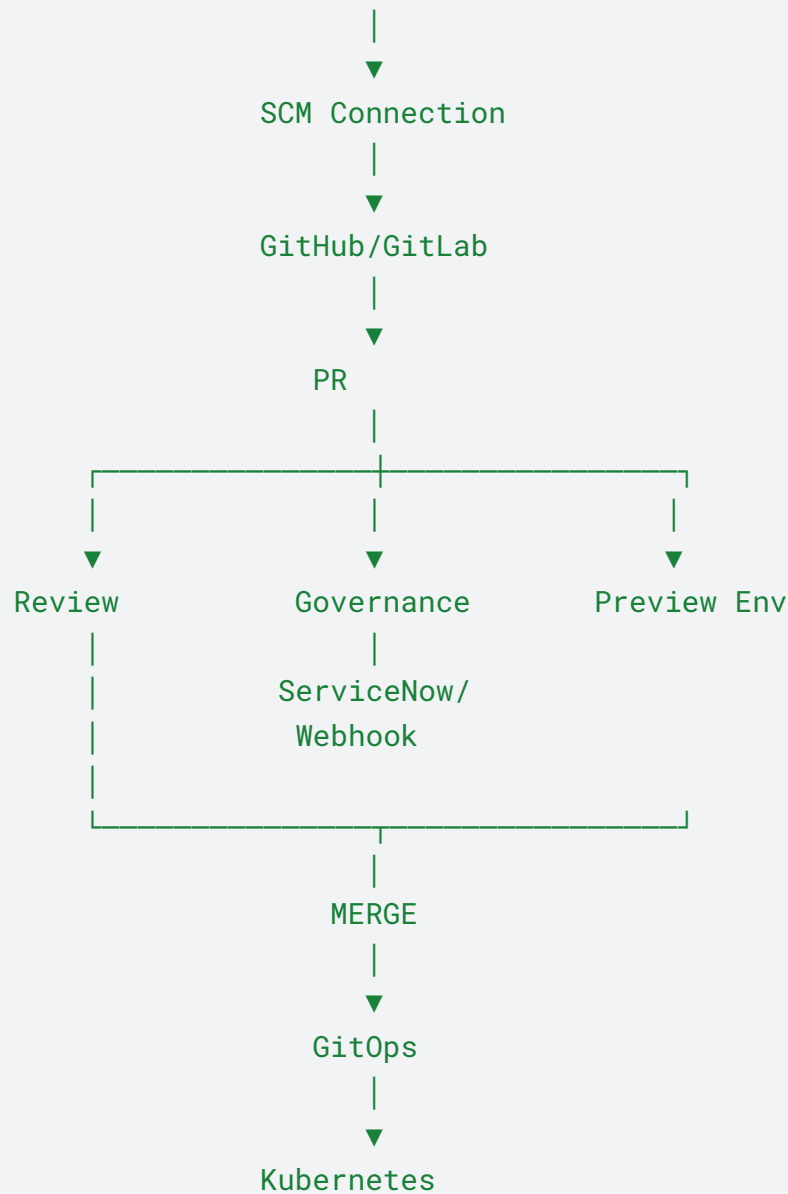
44. Complete Plural PR Automation architecture

Putting **everything you pasted** together:

None







45. The most important thing for Project 702

Now let's translate **Plural's** architecture into OpenChoreo terms.

Plural:

None

Plural Console



PrAutomation



context



Liquid/Lua/AI



Git



PR



GitOps

Your OpenChoreo project:

None

OpenChoreo Portal



Backstage Scaffold



Developer context



occ



OpenChoreo CRs



Git



PR



Flux CD / Argo CD



OpenChoreo Control Plane



Kubernetes

Your project proposal already defines this basic architecture: portal → Scaffold → conflict check → `occ` render → PR → GitOps → OpenChoreo → workloads. Project 702 Proposal_PR-Driven...

46. The strongest idea to take from Plural

Don't think of your feature as:

"We need to add a Create PR button."

That would be too simple.

Think of it as:

"We need to build a reusable self-service GitOps automation framework where a developer's intent is transformed into a validated Git change and submitted as a reviewable PR."

The complete model is:

